Stale Asynchronous Classification Results Re-Blurring Text After Watchlist Changes

Divyam Saluja
GitHub: salujadivyam

LinkedIn: salujadivyam

Abstract, After a title was switched off or marked finished in the popup, some text on the open page stayed blurred or became blurred again. The likely cause was a race between queued classification requests, created under the old watchlist and the later change of that watchlist. This report describes the race and the generation counter used to discard obsolete work.

Index Terms, race condition, asynchronous requests, state invalidation, Chrome extension.

I. Introduction

The content script queues text for classification, sends it to the API and blurs the element when the reply arrives. Each step is asynchronous, so work started under one configuration can finish after the configuration changed. This pattern is a form of race condition [1].

II. Problem Description

The developer reported that unticking a title or ticking Finished, did not fully remove blur, as shown in two screenshots in which some passages were unblurred and others were not. The storage change listener already removed the blur class from blurred elements and restarted the scan. Requests that were already queued or in flight still carried the old list of matched titles. When their replies arrived they applied the blur again. The explanation is inferred from the code path and from the pattern of the screenshots, since no trace of the individual requests was recorded.

III. Implemented Solution

A generation counter was added to the content script. It is incremented whenever the watchlist, the strict setting or the on/off switch changes and the queue is emptied at the same time. Each queued job stores the generation in which it was created. A job is dropped before it is sent if its generation is old and again after the reply arrives. The same restart routine clears the blur state of every element the script owns through a data flag, so a later click on text that is no longer tracked does nothing [2].

IV. Verification

The script passed a Node syntax check and the generation logic was reviewed line by line. A fake-page run confirmed that the normal path still blurs matching text. The race itself was not reproduced under automation, so the fix was handed to the developer for a retest.

V. Conclusion

Any queued work in the extension should carry a version of the state it was created under. Discarding results from an older version removes the class of defect, this symptom.

References

[1] MITRE, “CWE-362: Concurrent Execution Using Shared Resource with Improper Synchronization (‘Race Condition’).” Accessed: Oct. 28, 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/362.html

[2] Google, “chrome.storage,” Chrome for Developers. Accessed: Oct. 28, 2026. [Online]. Available: https://developer.chrome.com/docs/extensions/reference/api/storage